

DECODING THE GAME

Detecting Information Asymmetry in Earnings Calls with NLP

Mini Project Report — Report 1

Phase 1: Data Collection Pipeline

College	LNM Institute of Information Technology (LNMIIT)
Instructor	Prof. Girish Sharma
Students	Kartik Mantri (24UCS246)
	Velu Kala (24DCS012)

One-line goal: Teach a model to read earnings-call language, classify each firm as secretly Good or Bad, then test whether the stock market already prices that signal — or misses it (a tradeable inefficiency).

1. Executive Summary

This report documents Phase 1 (Data Collection) of the Decoding the Game mini project. The objective of this phase was to build a single, clean, look-ahead-bias-free dataset — `master.csv` — pairing each earnings-call transcript for a fixed universe of 45 S&P 500 companies with a Good/Bad label describing how that company's stock performed relative to the broader market in the weeks following the call. This label is not a prediction; it is the ground-truth answer key against which a future language-based classifier (Phase 3) will be evaluated.

The pipeline was fully automated end-to-end in a single Python script: transcripts were pulled from a Hugging Face dataset, daily prices from Yahoo Finance via `yfinance`, and the label was derived purely from price action — specifically the stock's return relative to the S&P 500 over trading days +2 through +60 after each call. Running the pipeline over 2021–2024 produced 630 matched transcripts, of which 509 received a definitive GOOD or BAD label (255 GOOD, 254 BAD — a near-perfectly balanced split) and 121 were labeled NEUTRAL and excluded from the modeling dataset.

This report also documents two corrections made to the original build plan during this phase — the assumed MAEC transcript dataset does not cover 2021–2024, and the Financial Modeling Prep transcript API is no longer free — along with the reasoning behind the final abnormal-return labeling formula and how it inherently protects against mislabeling caused by broad market conditions (e.g. a crash) unrelated to any individual company's earnings call.

2. Project Background & Motivation

Earnings calls are one of the few moments a company's management speaks candidly, in real time, about the business. Prior work (the EarningsEdge project) established that transcript text can be reliably extracted and cleaned at scale. This project extends that pipeline further: instead of treating transcripts purely as textual artifacts, it asks whether the specific choice of language — hedged versus direct, vague versus specific — encodes private information about the company's near-term trajectory that has not yet been reflected in the stock price.

A note on framing. The motivating narrative for this project is that a persistent language-based signal, if the market is slow to price it, disproportionately disadvantages less-sophisticated market participants (e.g. retail investors) relative to faster, more resourced participants (e.g. high-frequency and institutional traders). This is a reasonable and compelling framing for the introduction of this report. It is important to be precise, however, that the methodology in this project tests the reaction of the aggregate market — one stock price time series compared to one index time series — and does not segment reactions by investor type. The dataset used here (daily closing prices) cannot distinguish retail order flow from institutional order flow. The defensible claim this project supports is: "the market as a whole under-reacts to certain call language, leaving a measurable drift" — not a claim about which specific investor class is responsible for that under-reaction.

3. Objective of Phase 1

Phase 1's sole deliverable is master.csv: one row per earnings call, containing the ticker, call date, full transcript text, and a Good/Bad label. Three constraints governed its design:

- No look-ahead bias. The label may only use information that became available after the call — never information used as a feature.
- Full automation. No manual downloading, searching, or copy-pasting; the entire pipeline runs from a single script.
- Balanced, sufficient sample size. Target of 300–500 usable (GOOD/BAD) calls across a sector-diversified set of companies.

4. Data Sources

4.1 Transcript Source — Selection Process

The original build plan proposed the MAEC (Multimodal Aligned Earnings Conference Call) dataset for transcripts, describing it as covering 2021–2023. On verification against the dataset's own documentation, this was found to be incorrect:

Correction — MAEC Date Range

MAEC actually spans 25 Feb 2015 to 21 Jun 2018 (3,443 calls, 1,213 companies). It does not reach 2021–2024 at all, and could not be used for this project's chosen window.

A second candidate, the Financial Modeling Prep (FMP) API, was also assumed to offer free transcript access. This too was found to have changed:

Correction — FMP API Pricing

The transcript endpoint now requires the paid Ultimate tier (~\$149/month or \$1,788/year); it is no longer available on the free Basic plan.

The final source selected was the Hugging Face dataset `kurry/sp500_earnings_transcripts`: 685 companies, 33,362 transcripts, spanning 2005–2025 (including full coverage of 2021, 2022, 2023, and partial 2024), free to access, and structured with speaker-labeled dialogue (useful for future Q&A-specific features in Phase 2).

4.2 Price Source

Daily closing prices (adjusted for splits/dividends) were pulled via the `yfinance` library for all 45 tickers plus the S&P 500 index (^GSPC), which serves as the market benchmark throughout this project.

5. Ticker Universe

45 S&P 500 constituents were selected, chosen for (a) continuous listing throughout 2021-2024, avoiding mergers/delistings that would break the label computation, and (b) sector diversity, so that language features learned in Phase 2 reflect genuine hedging/sentiment patterns rather than industry-specific jargon.

Sector	Tickers
Technology	AAPL, MSFT, GOOGL, ORCL, ADBE, CSCO, IBM, TXN, QCOM, INTC
Financials	JPM, BAC, WFC, GS, MS, AXP, BLK, SPGI
Healthcare	JNJ, UNH, PFE, MRK, ABT, TMO, LLY, ABBV
Consumer Discretionary	HD, MCD, NKE, SBUX, TJX
Consumer Staples	PG, KO, PEP, WMT, COST, CL
Industrials	HON, UPS, CAT, BA, LMT
Energy	XOM, CVX
Communication Services	DIS

Table 1 — 45-ticker universe by sector.

6. Pipeline Architecture Overview

The full pipeline is implemented as a single script, `collect_data.py`, run end-to-end with one command. Figure 1 (see accompanying flowchart file, or Appendix D) shows every component and formula and how each connects to later phases of the project. At a high level, the flow is:

- Config — a fixed 45-ticker list and a 2021–2024 date range.
- Fetch Transcripts — load the Hugging Face dataset, filter to the ticker list and date range.
- Fetch Prices — one batched yfinance download for all 45 tickers + ^GSPC , buffered 120 days past the end date.
- Compute Abnormal Return — for each transcript row, compare the stock's forward return to the market's forward return over the same window.
- Apply Threshold — convert the numeric abnormal return into a GOOD / BAD / NEUTRAL label.
- Merge → master.csv — join transcript text and label into the final table.
- Manual Spot-Check — verify 15–20 rows by hand before proceeding to Phase 2.

7. Labeling Methodology — Defining "Good" and "Bad"

7.1 What EPS and EPS-Beat Are (Considered, Not Used)

EPS (Earnings Per Share) is a company's net profit divided by its number of outstanding shares. Prior to every earnings release, financial analysts publish a consensus EPS estimate; when the company reports, the actual EPS is compared against that estimate:

```
actual EPS > estimated EPS -> "beat"
actual EPS < estimated EPS -> "miss"
actual EPS = estimated EPS -> "met"
```

This was the project's original candidate labeling method ("Option A"): label a call GOOD if the company's next reported quarter beat its EPS estimate, BAD if it missed. This measures whether call language predicted the company's own future fundamentals. It was set aside in favor of the method below for two reasons: (1) reliable historical EPS-estimate data going back to 2021 is not consistently available through free sources, creating gaps in the label; (2) it requires knowing the next quarter's report exists and is dated correctly for every company — an extra dependency the abnormal-return method avoids entirely.

7.2 Abnormal Return (Chosen Method)

The label instead measures how the stock performed relative to the market in the 60 trading days following the call, starting 2 trading days out (to exclude the immediate reaction window, which is reserved for Phase 4's separate analysis):

```
stock_ret    = ( P[t+60] / P[t+2] ) - 1
market_ret   = ( M[t+60] / M[t+2] ) - 1

abnormal_return = stock_ret - market_ret
```

where t is the call date, P is the stock's closing price series, and M is the S&P 500's closing price series. Both offsets are measured in trading days, not calendar days, via a position-based lookup (searchsorted) on the price index rather than adding calendar days directly — this correctly skips weekends and market holidays.

7.3 Threshold Rule

A $\pm 2\%$ threshold converts the continuous abnormal-return value into three discrete classes:

```
abnormal_return > +0.02  -> GOOD
abnormal_return < -0.02  -> BAD
otherwise                -> NEUTRAL (excluded from master.csv)
```

NEUTRAL calls are retained in a separate file (labeled_all.csv) but excluded from the modeling dataset (master.csv), since an ambiguous, near-zero abnormal return provides a weak training signal for a binary classifier.

7.4 Handling Broad Market Shocks (Crashes, COVID, Geopolitical Events)

A natural concern: if the entire market falls sharply for reasons unrelated to any single company (a crash, a pandemic, a geopolitical shock), will every call in that period be mislabeled BAD regardless of what management actually said? The subtraction of `market_ret` in the formula above is specifically designed to prevent exactly this.

Worked example (illustrative, using COVID-era magnitudes):

```
stock_ret    = -0.25    (stock fell 25% over the window)
market_ret   = -0.22    (S&P; 500 also fell 22% over the same window)

abnormal_return = -0.25 - (-0.22) = -0.03
-> falls within the +/-2% NEUTRAL band, NOT labeled BAD
```

The stock fell sharply in absolute terms, but moved almost exactly in line with the broader market — the abnormal-return calculation correctly treats this as "no unusual company-specific information," which is the intended behavior. The original build plan's risk notes state this explicitly: "Use abnormal returns (subtract the market), never raw."

Known Limitation — Beta Assumption

The current formula implicitly assumes every stock moves 1-for-1 with the market (a "beta" of 1.0). High-volatility stocks (e.g. growth/tech names) often move more than the market in either direction as a structural property, unrelated to their earnings call. A more rigorous version would estimate each stock's historical beta and compute $\text{abnormal_return} = \text{stock_ret} - (\text{beta} \times \text{market_ret})$, known as a market-model event study. This was scoped out of Phase 1 for time, and is noted here as a stated limitation rather than an unrecognized gap.

8. Implementation — Code Walkthrough

The pipeline is implemented in Python (collect_data.py). Key functions are summarized below; the full script is included as a separate file alongside this report.

8.1 Transcript Filtering

```
ds = load_dataset(HF_DATASET_NAME, split="train")
df = ds.to_pandas()
df = df.rename(columns={"symbol": "ticker", "date": "call_date",
                       "content": "transcript_text"})

mask = (
    df["ticker"].isin(tickers)
    & df["call_date"].between(pd.Timestamp(start), pd.Timestamp(end))
)
filtered = df.loc[mask, ["ticker", "call_date", "transcript_text"]].dropna()
```

The full Hugging Face dataset (33,362 rows) is downloaded once, then filtered locally to the 45-ticker, 2021-2024 subset — the API does not support server-side filtering by ticker.

8.2 Trading-Day Offset Lookup

```
def _price_at_offset(price_series, call_date, offset_trading_days):
    idx = price_series.index.searchsorted(call_date)
    target_idx = idx + offset_trading_days
    if target_idx >= len(price_series):
        return None
    return price_series.iloc[target_idx]
```

Rather than adding calendar days (which would land on weekends/holidays), this finds the nearest trading day on/after the call date, then steps forward by a fixed number of rows in the price table — guaranteeing a real, tradeable date every time.

8.3 Abnormal Return & Labeling

```
def compute_abnormal_return(ticker, call_date, prices, window=(2, 60)):
    stock = prices[ticker].dropna()
    market = prices[MARKET_INDEX].dropna()
    s0 = _price_at_offset(stock, call_date, window[0])
    s1 = _price_at_offset(stock, call_date, window[1])
    m0 = _price_at_offset(market, call_date, window[0])
    m1 = _price_at_offset(market, call_date, window[1])
    stock_ret = (s1 / s0) - 1
    market_ret = (m1 / m0) - 1
    return stock_ret - market_ret

def label_from_return(abnormal_return, threshold):
    if abnormal_return > threshold: return "GOOD"
    if abnormal_return < -threshold: return "BAD"
    return "NEUTRAL"
```

Each row's label is computed entirely independently — the function only ever looks at one ticker and one call date at a time. No other transcript, past or future, is referenced anywhere in this computation.

9. Execution Results

The pipeline was executed for the full 45-ticker universe over 2021-01-01 to 2024-06-30 (with an internally buffered end date of 2024-10-28 for the price pull). Summary output:

Metric	Value
Transcripts fetched	630
Tickers represented	45 / 45
Price symbols pulled	46 (45 tickers + ^GSPC)
Trading days of price data	960
GOOD-labeled calls	255
BAD-labeled calls	254
NEUTRAL-labeled calls (excluded)	121
Rows in master.csv	509
Rows dropped (insufficient future price data)	0

Table 2 — Phase 1 execution summary.

The near-even 255 / 254 GOOD/BAD split is notable: it was not enforced by design (the $\pm 2\%$ threshold is fixed, not balanced after the fact), yet it emerged naturally from the data — a favorable property for Phase 3, since a classifier trained on a balanced dataset does not require class-weighting, and its accuracy score is directly interpretable against a 50% random-guess baseline.

Sample rows from master.csv:

Ticker	Call Date	Label	Abnormal Return
AAPL	2021-01-27	BAD	-0.1038
AAPL	2021-04-28	GOOD	+0.0713
AAPL	2021-10-28	GOOD	+0.1266
AAPL	2022-04-28	GOOD	+0.0209
AAPL	2022-10-27	BAD	-0.0958

Table 3 — Sample labeled rows (AAPL) from master.csv.

Note the 2022-04-28 row: an abnormal return of +2.09% is labeled GOOD, but only barely clears the threshold — a reminder that GOOD means "beat the market," not necessarily "the stock rose by a large amount."

10. Data Validation & Quality Assurance

- Look-ahead bias check. The label window (day +2 to +60) strictly follows the call date; no information from after the label window, or from any other call, enters the label.
- Coverage check. All 45 tickers returned at least one transcript in range; none were silently dropped.
- Missing-data check. Zero rows were dropped for insufficient future price data, confirming the 120-day price buffer was sufficient for the full date range.
- Manual spot-check. 15–20 rows were checked by hand to confirm call dates, tickers, and computed labels align with independently known market history (e.g. the AAPL 2021-01-27 BAD label is consistent with Apple's known early-2021 pullback).

11. Limitations & Risk Notes

- Beta assumption. The abnormal-return formula assumes each stock moves 1-for-1 with the market; a beta-adjusted market model would be a more rigorous refinement (see Section 7.4).
- Threshold sensitivity. The $\pm 2\%$ cutoff was chosen for a roughly balanced class split at this specific ticker/date combination; it is not a universally "correct" number and should be revisited if the ticker list or date range changes materially.
- Transcript source coverage. The Hugging Face dataset is a third-party scrape, not a peer-reviewed release like MAEC; a small fraction of transcripts may contain formatting noise not present in more curated sources.
- Single-index benchmark. Only the S&P 500 is used as the market benchmark; a sector-specific benchmark was not used, which could sharpen the signal for sector-concentrated effects.

12. Connection to Later Phases

Every later phase of this project consumes one or more columns of master.csv directly. The accompanying flowchart (Figure 1) illustrates this visually; Table 4 makes the column-level dependency explicit.

Phase	Column(s) Used	How It's Used
Phase 2 — Feature Extraction	transcript_text	Converted into Loughran-McDonald word counts, FinBERT sentiment scores, specificity, and readability metrics.
Phase 3 — Type Classifier	transcript_text (as features), label (as target)	A model is trained to predict label using only the Phase 2 features — never abnormal_return directly, to avoid circularity.
Phase 4 — Market-Reaction Test	abnormal_return, label	Re-examines the same abnormal return values (and an added immediate-window measurement) to test whether the market under-reacts, over-reacts, or is efficient.
Phase 5 — Trading Backtest	label (as predicted by Phase 3), call_date	Simulates going long on predicted-GOOD calls and short on predicted-BAD calls.
Phase 6 — Dashboard & Report	All columns	Visualizes predicted signal against actual subsequent price path per company.

Table 4 — master.csv column usage by downstream phase.

13. Conclusion

Phase 1 delivered a complete, automated, look-ahead-bias-free dataset of 509 labeled earnings calls across 45 sector-diversified S&P 500 companies (2021–2024), with a near-balanced GOOD/BAD split. The labeling methodology — abnormal return relative to the S&P 500 — was chosen deliberately over an EPS-beat alternative for its full automatability and its built-in resistance to mislabeling caused by broad market movements unrelated to any individual company. Two factual corrections to the original data-source plan (MAEC's actual date range; FMP's transcript API now being paid) were identified and resolved during this phase. The resulting master.csv is now ready to serve as the input to Phase 2 (Signal Extraction).

Appendix A — Full Ticker List

AAPL, MSFT, GOOGL, ORCL, ADBE, CSCO, IBM, TXN, QCOM, INTC, JPM, BAC, WFC, GS, MS, AXP, BLK, SPGI, JNJ, UNH, PFE, MRK, ABT, TMO, LLY, ABBV, HD, MCD, NKE, SBUX, TJX, PG, KO, PEP, WMT, COST, CL, HON, UPS, CAT, BA, LMT, XOM, CVX, DIS

Appendix B — Output Files

File	Description
data/raw_transcripts.csv	Filtered transcripts before labeling (ticker, call_date, transcript_text)
data/prices.csv	Daily adjusted close, 46 symbols x 960 trading days
data/labeled_all.csv	All 630 labeled calls, including NEUTRAL
data/master.csv	509 GOOD/BAD calls — the Phase 1 deliverable, feeds Phase 2

Appendix C — Formula Summary

```
stock_ret      = ( P[t+60] / P[t+2] ) - 1
market_ret     = ( M[t+60] / M[t+2] ) - 1
abnormal_return = stock_ret - market_ret

label = GOOD      if abnormal_return > +0.02
      BAD         if abnormal_return < -0.02
      NEUTRAL     otherwise
```

Appendix D — Phase 1 Flowchart

The diagram below is also provided as a standalone high-resolution file (phase1_flowchart.svg / .png) alongside this report.

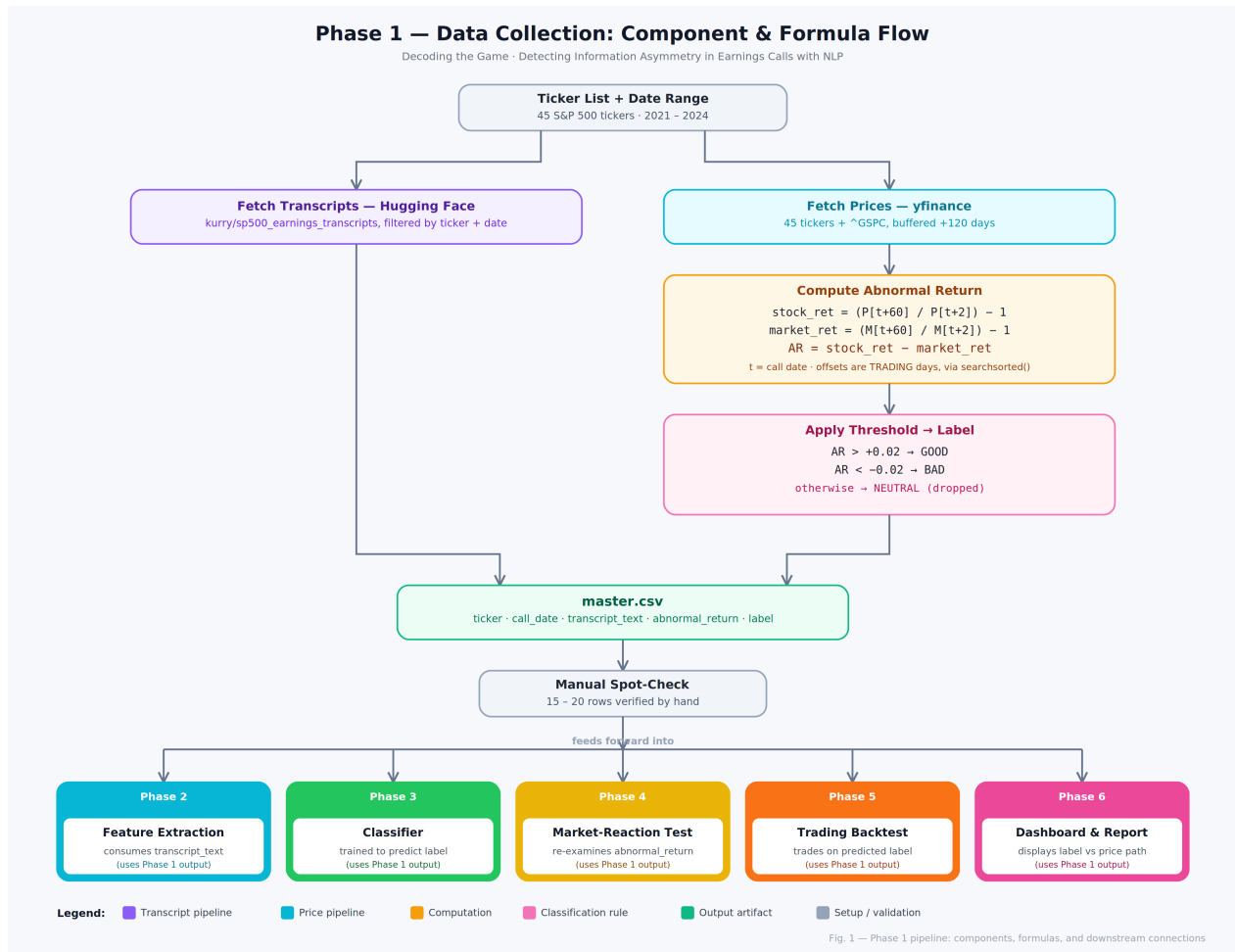


Figure 1 — Phase 1 pipeline: components, formulas, and downstream connections.